

CS3002E Compiler Design

Monsoon 2025

Lecture #8

Top-Down Parsing-1

Department of Computer Science & Engineering
NIT Calicut

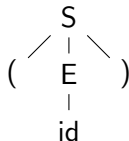
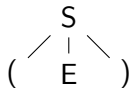
Top-Down Parsing

- Builds (or simulates the steps in building) the parse tree, starting from the root and creating nodes in preorder
- Equivalent to doing a Left-most derivation

Top-Down Parsing

$$S \rightarrow (E)$$
$$E \rightarrow id$$

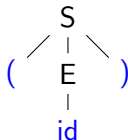
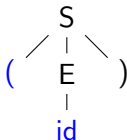
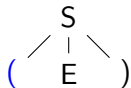
S



Action at a tree node

- Nonterminal - choose production, add children
- Terminal - match with the next input token

Top-Down Parsing

$$S \rightarrow (E)$$
$$E \rightarrow id$$


Input: (matched tokens are shown in blue color)

(_↑id)

(id_↑)

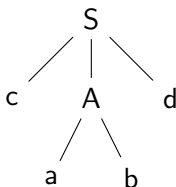
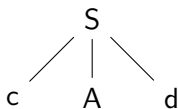
(id)_↑

Top-Down Parsing

$$S \rightarrow cAd$$

$$A \rightarrow ab$$

Input: cabd



Parse:

1. cad
2. dca
3. cabdb

Recursive-Descent Parsing

A procedure defined for each Nonterminal

```
S(){  
    match('c');  
    A();  
    match('d');  
}
```

```
A(){  
    match('a');  
    match('b');  
}
```

match(x) - compares *x* with the **look-ahead symbol** (the next input symbol)

- if both match, advances the input pointer to the next input symbol
- if mismatch, reports error and stops

Recursive-Descent Parsing

$$S \rightarrow (A)$$

$$A \rightarrow ab \mid b$$

Parse: (b)

At node A, parser can choose either

$$A \rightarrow b$$

$$A \rightarrow ab$$

Recursive-Descent Parsing

$$S \rightarrow (A)$$
$$A \rightarrow ab \mid b$$

Parse: (b)

At Node A,

if $A \rightarrow b$ is chosen, parsing is successful

what if $A \rightarrow ab$ is chosen?

Recursive-Descent with backtracking

$$S \rightarrow (A)$$
$$A \rightarrow ab \mid b$$

Parse: (b)

At Node A

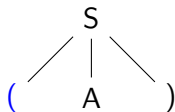
if $A \rightarrow b$ is chosen, parsing is successful

if $A \rightarrow ab$ is chosen:

mismatch in the next step

can **backtrack** and try the other production, $A \rightarrow b$

Top-Down Parsing - choosing a production



Input: (↑a b)

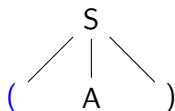
At node A, can the parser decide whether to choose

$A \rightarrow b$

or

$A \rightarrow ab$?

Top-Down Parsing - choosing a production



Input: (↑a b)

At node A, can the parser decide whether to choose

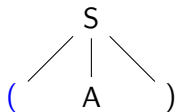
$A \rightarrow b$

or

$A \rightarrow ab$

based on the next input symbol?

Predictive Parsing



choose production $A \rightarrow b$ or $A \rightarrow ab$ based on the **look-ahead** (next input symbol)

if look-ahead is **a**, choose $A \rightarrow ab$

if look-ahead is **b**, choose $A \rightarrow b$

Parser **predicts** the production based on the **look-ahead symbol**.
Avoids backtracking.

Predictive Parsing

Procedure for A

```
A(){  
    switch(lookahead){  
        case 'a':  match('a'); match('b'); break;  
        case 'b':  match('b'); break;}}
```

Predictive Parsing

$$A \rightarrow \alpha \mid \beta$$

α begins with a terminal a (α is $a\alpha'$)

β begins with a terminal b (β is $b\beta'$)

If lookahead is a choose $A \rightarrow \alpha$, if lookahead is b choose $A \rightarrow \beta$

Predictive Parsing: Exercise

Given the grammar:

$$\begin{array}{lcl} stmt & \rightarrow & \mathbf{id} = expr \\ & | & \mathbf{if} \ (\mathbf{id}) \ stmt \\ & | & \mathbf{while} \ (\mathbf{id}) \ stmt \end{array}$$
$$\begin{array}{lcl} expr & \rightarrow & \mathbf{id} \\ & | & \mathbf{num} \end{array}$$

Show the actions of a predictive parser for the following inputs:

1. `sum=x1`
2. `if (x) length=0`

Predictive Parsing

$$S \rightarrow (A)$$
$$A \rightarrow C \mid D$$
$$C \rightarrow ab$$
$$D \rightarrow b$$

Does look-ahead help in choosing the production for A?

Predictive Parsing

$$S \rightarrow (A)$$

$$A \rightarrow C \mid D$$

$$C \rightarrow ab$$

$$D \rightarrow b$$

Does look-ahead help in choosing the production for A?

- if lookahead is a , choose $A \rightarrow C$
- if lookahead is b , choose $A \rightarrow D$

Choice based on the strings derivable from C and D

based on the terminal symbols that can begin strings derivable from C and D

.

FIRST

Define **FIRST**(α), where α is any string of grammar symbols, to be the set of terminal symbols that can begin strings derivable from α .

FIRST

$$\text{FIRST}(ab) = \{a\}$$

$$\text{FIRST}(b) = \{b\}$$

FIRST

$$A \rightarrow \alpha \mid \beta$$

α begins with a terminal a (α is $a\alpha'$)

β begins with a terminal b (β is $b\beta'$)

$$a \in FIRST(\alpha)$$

$$b \in FIRST(\beta)$$

FIRST

$$S \rightarrow aAb$$

$$FIRST(S) = FIRST(aAb) = \{a\}$$

Predictive Parsing

$S \rightarrow aAb$

$A \rightarrow B \mid C$

$B \rightarrow bd$

$C \rightarrow c$

$FIRST(A) = ?$

Predictive Parsing

$S \rightarrow aAb$

$A \rightarrow B \mid C$

$B \rightarrow bd$

$C \rightarrow c$

$$FIRST(A) = FIRST(B) \cup FIRST(C) = \{b, c\}$$

Predictive Parsing

$$A \rightarrow \alpha \mid \beta$$

if lookahead $\in FIRST(\alpha)$ choose $A \rightarrow \alpha$

if lookahead $\in FIRST(\beta)$ choose $A \rightarrow \beta$

Predictive Parsing

$$A \rightarrow \alpha \mid \beta$$

if lookahead $\in FIRST(\alpha)$ choose $A \rightarrow \alpha$

if lookahead $\in FIRST(\beta)$ choose $A \rightarrow \beta$

$$A \rightarrow aB \mid aC$$

$FIRST(A)?$

Predictive Parsing

$$A \rightarrow \alpha \mid \beta$$

If $FIRST(\alpha)$ and $FIRST(\beta)$ are disjoint, a parsing decision can be made based on the lookahead symbol.

Computing FIRST¹

- For a terminal a , $FIRST(a) = \{a\}$
- For a string α of the form $a\alpha'$, $FIRST(\alpha) = \{a\}$
- For a Nonterminal A , for every production $A \rightarrow \alpha$, add every symbol in $FIRST(\alpha)$ to $FIRST(A)$

¹assuming no ϵ productions

Computing FIRST

$S \rightarrow aAb$

$A \rightarrow B \mid C$

$B \rightarrow bd$

$C \rightarrow c$

$FIRST(S) = FIRST(aAb) = \{a\}$

$FIRST(B) = \{b\}$

$FIRST(C) = \{c\}$

$FIRST(A) = FIRST(B) \cup FIRST(C) = \{b, c\}$

Predictive Parsing: Exercise

Given the grammar:

$$\begin{array}{lcl} \textit{stmt} & \rightarrow & \textbf{id}=\textit{expr} \\ & | & \textbf{if (id) stmt} \\ & | & \textbf{while (id) stmt} \end{array}$$
$$\begin{array}{lcl} \textit{expr} & \rightarrow & \textbf{id} \\ & | & \textbf{num} \end{array}$$

Compute FIRST set for each string that appears as the body of a production. Show how these sets help the parser in making parsing decisions for the following inputs.

1. `sum=x1`
2. `if (x) length=0`

Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Chapter 4, Sections 4.4.